1. Abschlussprojekt

Julian Huber
MCI

Peter Kandolf
LFU Innsbruck

Matthias Panny
MCI

← Back to overview

Abschlussprojekt SS 2026

Allgemeines zum Abschlussprojekt (1/2)

- Gruppenarbeit mit **2 Personen**
- Versionsverwaltung & kollaboratives Arbeiten mit git
- Umsetzen des Projektes, welches wir in diesen Einheiten besprechen werden
- Abgabetermin: **16.07.2026** bzw. **23.07.2026** um **23:55 Uhr**

Umfang des Abschlussprojektes

- **35h** pro Person * 2 Personen pro Gruppe = **70h** pro Gruppe
- 35h pro Person setzt sich aus folgenden Annahmen zusammen:
  - gemeinsame LV: 60UE á 45min = 45h
  - Vor- & Nachbereitung inkl. 14 HÜ: 1,5h pro 2 UE = 45h
  - Kursumfang: 5 ECTS = 125h
  - 125h - 45h - 45h = 35h

Allgemeines zum Abschlussprojekt (2/2)

Projektnote

- In den nächsten Folien wird ein Projekt inkl. minimaler Anforderungen vorgestellt
- Diese minimale Umsetzung entspricht **60%** der Gesamtpunkte
- Die restlichen **40%** können durch Erweiterungen erreicht werden

Gesamtnote

- Die Gesamtnote setzt sich aus der Projektnote (50%), der Klausurnote (50%) und +10% Bonuspunkten wenn alle Hausübungen abgegeben wurden, zusammen.

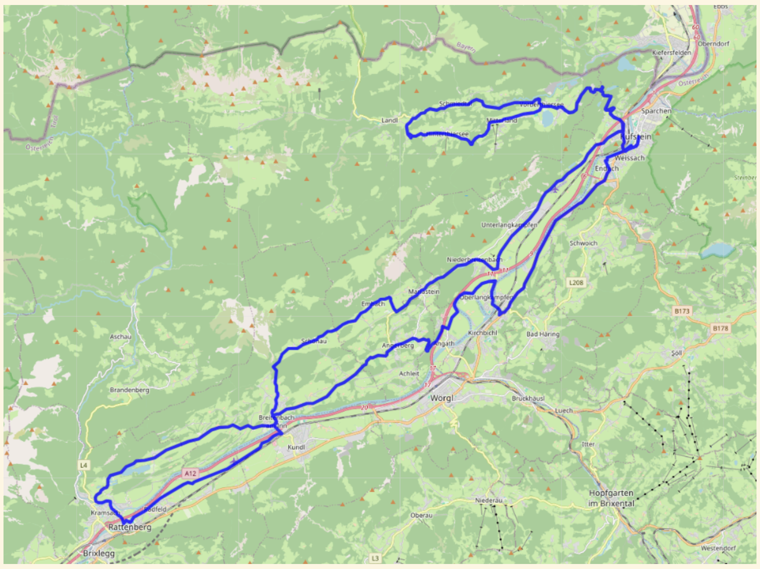
```python
def calculate_final_grade(project_grade: float, exam_grade: float, all_homeworks_submitted: bool) -> float:
    assert 0 <= project_grade <= 100, "Project grade must be between 0 and 100"
    assert 0 <= exam_grade <= 100, "Exam grade must be between 0 and 100"

    final_grade = 0.5 * project_grade + 0.5 * exam_grade
    if all_homeworks_submitted:
      final_grade += 10

    return min(final_grade, 100)  # Maximal 100% erreichen
```

Aufgabenstellung

- Eine `Python`-Anwendung zur Auslegung eines E-Bikes basierend auf realen Daten[1]
- Ein Datensatz an GPS-Daten mit Zeitstempeln wird bereit gestellt, woraus Geschwindigkeit, Beschleunigung, Leistung, Steigung, etc. berechnet werden können
- Daraus kann die benötigte Batteriekapazität, die Motorleistung, etc. berechnet werden



Route aus den GPS-Daten

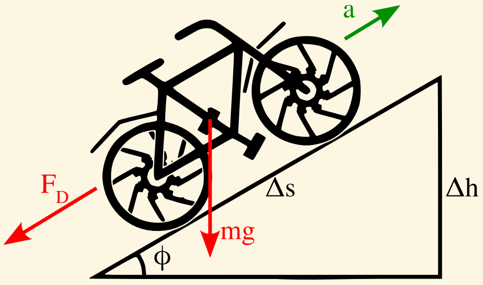
[1] Alternativen zu diesem Projekt werden zum Schluss diskutiert

Auswertung der Daten

Was kann wie bestimmt werden?

Berechnung der Geschwindigkeit, Beschleunigung, Leistung, etc. aus den GPS-Daten

- Die GPS-Daten bestehen aus Zeitstempeln, Längen- und Breitengraden sowie Höheninformationen
- Aus den Längen- und Breitengraden kann die zurückgelegte Strecke z.B. mit der Haversine-Formel berechnet werden



- Daraus können Geschwindigkeit und Beschleunigung bestimmt werden → hierbei gilt $v = \frac{\mathrm{d}s}{\mathrm{d}t} \approx \frac{\Delta s}{\Delta t}$ für die kleinen Zeitintervalle Δt zwischen den GPS-Messungen
- Die Höheninformation kann genutzt werden, um die Steigung der Strecke zu bestimmen
- Aus Geschwindigkeit v, Beschleunigung a und Steigung φ kann über das Freikörperdiagramm rechts die benötigte Kraft bestimmt werden die das E-Bike antreibt. Hierbei ist F_D die Luftwiderstandskraft die sich aus $F_D = \frac{1}{2} \rho\, c_W\, A\, v^2$ berechnet

Berechnung des Motorstroms

- Aus der notwendigen Antriebskraft F kann sowohl direkt die Leistung als auch das notwendige Drehmoment am Rand T bestimmt werden → hierbei ist zu beachten, dass nur ein Rad angetrieben wird
- Für unser vereinfachtes Modell des Antriebsstranges nehmen wir an, dass der Antrieb

über einen Radnabenmotor ohne Getriebe erfolgt

- Weiters ist anzunehmen, dass das Drehmoment T direkt proportional zum Motorstrom I_{mot} ist → die Proportionalitätskonstante ist die sogenannte Motorkonstante K_m (in $\mathrm{Nm/A}$)

Anforderungen, Erweiterungen & Abgabe

Was ist unbedingt zu erfüllen?

Allgemeine Minimalanforderungen

- Softwareprojekt in **Python** mit **Objektorientierung**
- Versionskontrolle über **git** & **GitHub**
- Anwendung zur Auswertung der Daten:
  - Möglichkeit zur Simulation der Fahrt mit einem einfachen Akku-Modell
- Software-Dokumentation:
  - `requirements.txt`-Datei mit allen packages
  - `README.md` im Repository mit Anleitung zur Installation und Ausführung
- Projekt-Dokumentation - Zwei Möglichkeiten
  - `README.md` ergänzen um umgesetzten Erweiterungen, UML-Diagrammen der Softwarestruktur, Quellen zu verwendeten Inhalten etc.
  - *ODER*

  - Kurzer **Bericht als pdf-Dokument** mit selbem Inhalt

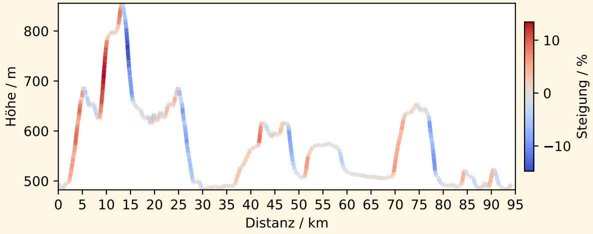
Spezifische Minimalanforderungen (1/2)

- Eine `Python`-Anwendung soll entwickelt werden
- Darin können die GPS-Daten eingelesen und ausgewertet werden
- Es soll folgendes daraus bestimmt werden:
  - Geschwindigkeit
  - Beschleunigung
  - Leistung
  - Steigung
  - Drehmoment am Motor
  - Motorstrom bei bekannter Motorkonstante
  - Entwicklung des Ladezustands des Akkus über die Fahrt
- Aus dem gesamten Datensatz können ebenso folgend Kenngrößen bestimmt werden:
  - Durchschnittsgeschwindigkeit
  - Zurückgelegte Strecke
  - Benötigte Zeit
  - Höhenmeter Anstieg und Abstieg
  - Maximalleistung

Spezifische Minimalanforderungen (2/2)

- Für die Simulation der Akkus soll das grundlegende Modell aus 09 | 2. Einführung in die OOP - Übung verwendet werden
- Es sind **zwei** verschiedene Akkutypen zu simulieren, welche sich in ihren Kennlinien unterscheiden → sind nachfolgend beschrieben
- Sie sollen die Ergebnisse für ihre Simulation grafisch darstellen:
  - Geschwindigkeit, Leistung, Ladezustand, etc. über die Zeit
  - Höhenprofil der Fahrt
  - etc.
- Die Simulation soll mit Logging und Fehlerbehandlung ausgestattet sein, um z.B. zu verhindern, dass der Ladezustand über 100% oder unter 0% fällt
- Der gesamte Ablauf der Simulation muss in einem Aktivitätsdiagramm dargestellt werden
- Das Projektverzeichnis muss eine `requirements.txt`-Datei mit allen benötigten Paketen enthalten → das README dokumentiert die Installation und Ausführung des Projekts

Mögliche Erweiterungen (1/2)



- Plotten von diversesten hilfreichen Darstellungen
- Alle commit-messages im `github`-Repository sollen dem Konzept der Conventional Commits folgen, um die Nachvollziehbarkeit der Entwicklung zu gewährleisten
- Plotten der Strecke auf einer Karte (z.B. mit `folium` oder `geopandas`)
- Unit-Tests für sinnvolle Teile der Software
- Durchführen von automatischen Parameterstudien mit:
  - Reifendurchmesser, Masse des Fahrers/Fahrrads, Luftwiderstandsbeiwert, etc.

Mögliche Erweiterungen (2/2)

- Bestimmung der Luftdichte aus Temperatur und Höhe über dem Meeresspiegel
- Simulation des Rollwiderstands
- Simulation der Akkutemperatur und deren Einfluss auf die Akkuleistung
- Simulation eines Bremswiderstands, der “überschüssige” Energie dissipiert, wenn sie nicht mehr in den Akku geladen werden kann
- Nutzen einer API um die GPS-Koordinaten in Adressen/Orte umzuwandeln (Reverse Geocoding) und diese in Plots darzustellen
- Aufrufen von Wetterdaten für die Zeitpunkte der Fahrt und Berücksichtigung von Wind, etc.
- Bestimmung der Orientierung (Himmelsrichtung) aus den GPS-Daten
- Generieren eines Reports über die Fahrt mit den wichtigsten Kenngrößen, Plots, etc. → z.B. als $\LaTeX$-Dokument

Abgabe

- Abgabetermin: **16.07.2026** bzw. **23.07.2026** um **23:55 Uhr**
- Alle Gruppenmitglieder müssen den Link zum Repository auf Sakai abgeben
- Wenn die zwei Gruppenmitglieder unterschiedlichen LV-Gruppen (2A bzw. 2B) angehören, so gilt der **frühere** Abgabetermin für beide Mitglieder
- Der Beitrag der einzelnen Mitglieder wird anhand ihrer Commits beurteilt → jedes Mitglied soll **aktiv** und **inhaltvoll** am Projekt mitarbeiten
- Es wird der Stand des letzten Commits im `main` bzw. `master`-Branch, der vor der Deadline stattgefunden hat, bewertet

- Wenn ein Projekt nicht mit Hilfe der Installationsanleitung im `README.md` oder Bericht ausführbar ist, so wird es mit **0 Punkten** bewertet

Alternatives Abschlussprojekt

- Falls die Aufgabenstellung einer Gruppe nicht zusagt, kann ein alternatives Abschlussprojekt mit dem jeweiligen Vortragenden vereinbart werden
- In der ersten Hälfte des letzten Termins der LV muss diese Aufgabenstellung mit dem Vortragenden vereinbart werden → anschließend wird sie mit dem Vortragenden der anderen Gruppe abgestimmt

Was muss vereinbart werden?

- Idee des eigenen Projektes
- Minimalanforderungen → Programmiersprache, Versionsverwaltung, Dokumentation, etc. bleibt immer gleich
- Mögliche Erweiterungen
- Diese sind dann **verbindlich** für die jeweilige Gruppe → es kann aber immer auf das allgemeine Projekt zurück gewechselt werden

“Simulations-Daten”

Parameter und fiktive Datenblätter für die zu simulierenden Akku-Typen

Fahrrad

- Masse Fahrer: $70\ \mathrm{kg}$
- Masse Fahrrad: $10\ \mathrm{kg}$
- Produkt aus c_W -Wert und Stirnfläche A: $0.5625\ \mathrm{m}^2$
- Raddurchmesser: $27\ \mathrm{inch}$
- Motorkonstante: $1.5\ \mathrm{Nm/A}$

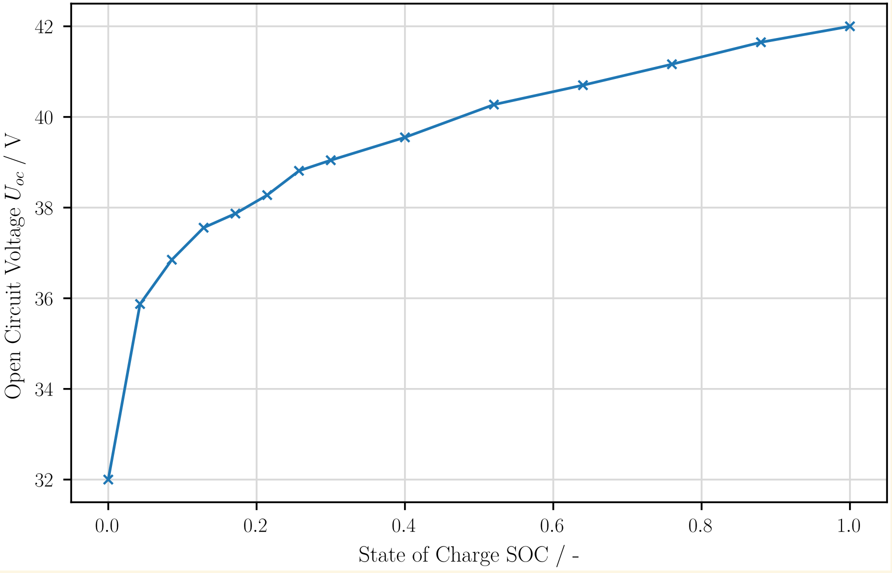
LiPo-Akku - Grundlegend Aufbau

- Aufbau: 10SxP (10 Zellen in Serie, x Zellen in Parallel)
- Nennspannung: $3.7\ \mathrm{V}$ pro Zelle
- Minimale Spannung: $3.2\ \mathrm{V}$ pro Zelle
- Maximale Spannung: $4.2\ \mathrm{V}$ pro Zelle
- Innenwiderstand $8\ \mathrm{m\Omega}$ pro Zelle

LiPo-Akku - Kennlinien

- Kennlinie der Open-Circuit-Spannung (OCV) in Abhängigkeit von der State of Charge (SOC)

SoC / –	U_{oc} / V
0.00	32.00
0.04	35.87



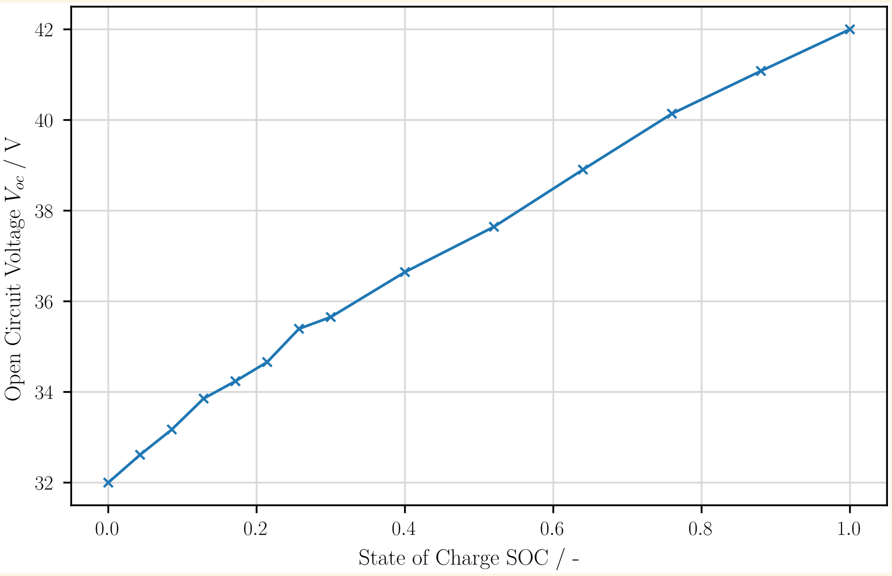
0.09	36.85
0.13	37.56
0.17	37.87
0.21	38.28
0.26	38.81
0.30	39.05
0.40	39.55
0.52	40.27
0.64	40.70
0.76	41.16
0.88	41.65
1.00	42.00

NMC-Akku - Grundlegend Aufbau

- Aufbau: 10SxP (10 Zellen in Serie, x Zellen in Parallel)
- Nennspannung: $3.7\ \mathrm{V}$ pro Zelle
- Minimale Spannung: $3.2\ \mathrm{V}$ pro Zelle
- Maximale Spannung: $4.2\ \mathrm{V}$ pro Zelle
- Innenwiderstand $7\ \mathrm{m\Omega}$ pro Zelle

NMC-Akku - Kennlinien

- Kennlinie der Open-Circuit-Spannung (OCV) in Abhängigkeit von der State of Charge (SOC)



SoC / –	U_{oc} / V
0.00	32.00
0.04	32.61
0.09	33.17
0.13	33.85
0.17	34.24
0.21	34.66
0.26	35.39
0.30	35.65
0.40	36.65
0.52	37.64
0.64	38.91
0.76	40.14
0.88	41.08
1.00	42.00